Question 1 | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *A = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }

    int k;
    if (scanf("%d", &k) != 1) {
        free(A);
        return 0;
    }

    int i = 0, j = 1, found = 0;

    while (i < n && j < n) {
        if (i == j) {
            j++;
        } else if (A[j] - A[i] == k) {
            found = 1;
            break;
        } else if (A[j] - A[i] < k) {
            j++;
        } else {
            i++;
        }
    }

    printf("%d\n", found);

    free(A);
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *A = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }

    int k;
    if (scanf("%d", &k) != 1) {
        free(A);
        return 0;
    }

    int i = 0, j = 1, found = 0;

    while (i < n && j < n) {
        if (i == j) {
            j++;
        } else if (A[j] - A[i] == k) {
            found = 1;
            break;
        } else if (A[j] - A[i] < k) {
            j++;
        } else {
            i++;
        }
    }

    printf("%d\n", found);

    free(A);
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

int main()
{
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int ascending(const void *a, const void *b)
{
    return (*(int *)a - *(int *)b);
}

int descending(const void *a, const void *b)
{
    return (*(int *)b - *(int *)a);
}

int main()
{
    int n;
    scanf("%d", &n);

    int a[n], b[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &b[i]);
    }

    // Sort first array in ascending order
    qsort(a, n, sizeof(int), ascending);

    // Sort second array in descending order
    qsort(b, n, sizeof(int), descending);

    long long sum = 0;

    for (int i = 0; i < n; i++)
    {
        sum += (long long)a[i] * b[i];
    }

    printf("%lld", sum);

    return 0;
}
```

	Input	Expected	Got	
✔	3 1 2 3 4 5 6	28	28	✔
✔	4 7 5 1 2 1 3 4 1	22	22	✔
✔	5 20 10 30 10 40 8 9 4 3 10	590	590	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct Mark 1.00 out of 1.00

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int findPair(int arr[], int left, int right, int x)
{
    if (left >= right)
        return 0;

    int sum = arr[left] + arr[right];

    if (sum == x)
    {
        printf("%d\n", arr[left]);
        printf("%d\n", arr[right]);
        return 1;
    }

    if (sum < x)
        return findPair(arr, left + 1, right, x);
    else
        return findPair(arr, left, right - 1, x);
}

int main()
{
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    int x;
    scanf("%d", &x);

    if (!findPair(arr, 0, n - 1, x))
    {
        printf("No");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course